

Hall Renting Service

Сервіс для керування та бронювання конференц-залів. Пропонується можливість додавання, редагування, видалення конференц-залів, а також їхніх послуг. Також є можливість пошуку доступних для бронювання залів та, власне, бронювання залів. Для взаємодії із сервером надається Swagger та Scalar.

Endpoints

1. HallService:

- a. GET /api/hall-services?ids="<guid>"&ids="<guid>"&...: Повертає загальний список послуг. Якщо не вказувати ids, то повертаються усі можливі послуги, інакше – вказані послуги.
- b. POST /api/hall-service; Body:

```
{
  "id": "<guid>",
  "name": "<string>"
}
```

Додає послугу до загального списку послуг. Аргумент для “name” повинен бути унікальним. Аргумент “id” ігнорується, але повинен бути у форматі guid. Повертається новий guid, який був присвоєний для створеної послуги.
- c. PUT /api/hall-service; Body:

```
{
  "id": "<guid>",
  "name": "<string>"
}
```

Оновлює послугу. Аргумент для “name” повинен бути унікальним. Послуга із аргументом “id” повинна існувати в базі даних.
- d. DELETE /api/hall-services?ids="<guid>"&ids="<guid>"&...: Видаляє послуги із загального списку. Якщо не вказати ids, то видалиться увесь список, інакше – вказані послуги.

2. Hall:

- a. GET /api/halls?ids="<guid>"&ids="<guid>"&...: Повертає загальний список залів. Якщо не вказувати ids, то повертаються усі зали, інакше – вказані.
- b. POST /api/hall; Body:

```
{
  "id": "<guid>",
  "name": "<string>",
  "capacity": <number>,
  "pricePerHour": <number>,
  "hallServices": [
    {
      "hallServiceId": "<guid>",
      "price": <number>
    }
  ],
  "bookings": []
}
```

Додає конференц-зал та створює зв'язок між ним та послугами. Аргумент “id” ігнорується, але повинен бути у форматі guid. Аргумент “capacity” повинен бути більше 0. Аргумент “pricePerHour” повинен бути не від’ємним. Список “hallServices” повинен містити “hallServiceId” реальних послуг, а “price” повинен бути не

від'ємним. Аргумент "bookings" ігнорується. Повертається новий guid, який був присвоєний для створеного залу.

c. PUT /api/hall-service; Body:

```
{
  "id": "<guid>",
  "name": "<string>",
  "capacity": <number>,
  "pricePerHour": <number>,
  "hallServices": [
    {
      "hallServiceId": "<guid>",
      "price": <number>
    }
  ],
  "bookings": []
}
```

Оновлює конференц-зал та зв'язки між ним та послугами. Аргумент "id" повинен відповідати залу, який оновлюється. Аргумент "capacity" повинен бути більше 0. Аргумент "pricePerHour" повинен бути не від'ємним. Список "hallServices" повинен містити "hallServiceId" реальних послуг, а "price" повинен бути не від'ємним. Аргумент "bookings" ігнорується.

d. DELETE /api/halls?ids="<guid>"&ids="<guid>"&...: Видаляє зали, його зв'язки із послугами та бронювання із загального списку. Якщо не вказати ids, то видалиться увесь список, інакше – вказані зали.

3. Booking:

a. GET /api/bookings?ids="<guid>"&ids="<guid>"&...: Повертає загальний список бронювань. Якщо не вказувати ids, то повертаються усі бронювання, інакше – вказані.

b. GET

/api/halls/search?bookingStart="<datetime>"&bookingDuration="<timespan>"&capacity="<number>": Повертає список залів, які не мають бронювань в періоді між "bookingStart" та "bookingStart + bookingDuration", а також їхнє "capacity" >= вказаної. Аргумент "bookingStart" повинен бути >= теперішнього часу. Аргумент "bookingDuration" повинен бути більше "00:00:00". Аргумент "capacity" повинен бути не від'ємним.

c. POST /api/hall/book; Body:

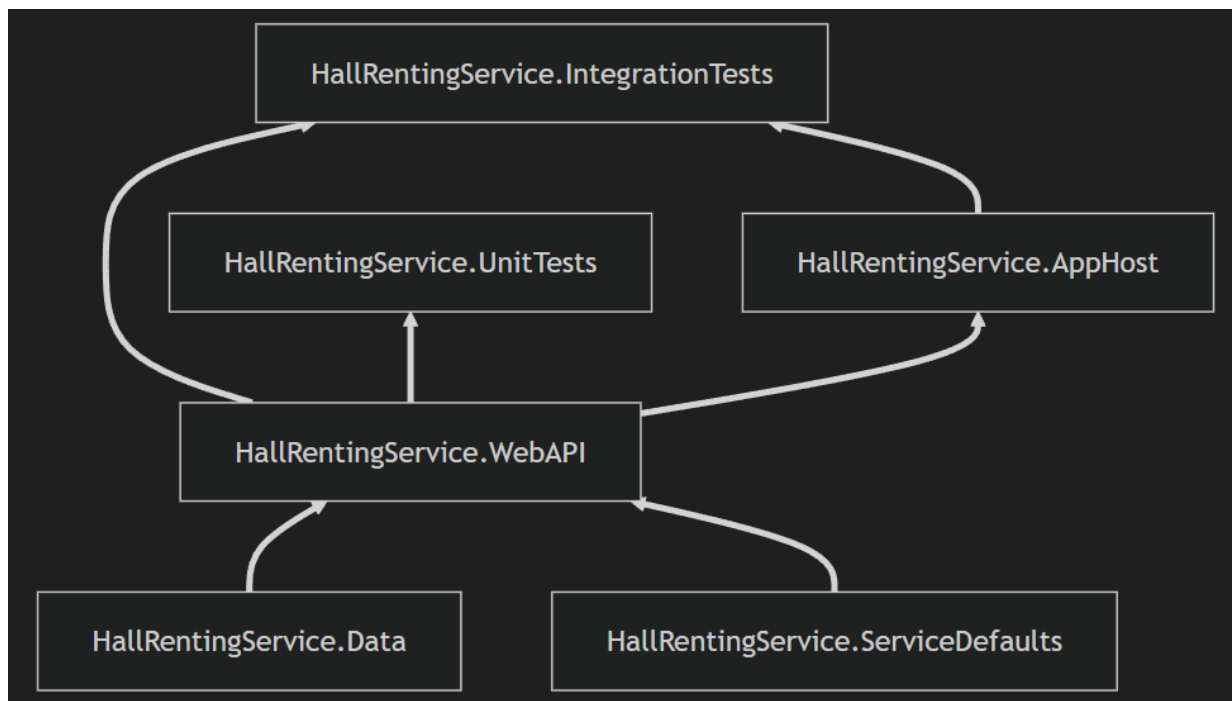
```
{
  "hallId": "<guid>",
  "bookingStart": "<datetime>",
  "bookingDuration": "<timespan>",
  "hallServices": [
    "<guid>"
  ]
}
```

Бронює зал, якщо період бронювання не перетинається із іншими. Аргумент "hallId" повинен бути реальним id залу. Аргумент "bookingStart" повинен бути >= теперішнього часу. Аргумент "bookingDuration" повинен бути більше "00:00:00". Список "hallServices" повинен містити id сервісів, які доступні для обраного залу (hallId). В результаті бронювання вираховується ціна (враховуючи знижки/націнки + послуги) та повертається пара (Id, TotalPrice), де "Id" – присвоєний ідентифікатор бронювання, а "TotalPrice" – фінальна ціна бронювання.

Пояснення:

1. GET запити повертають всю потрібну інформацію, натомість інші endpoints можуть повертати тільки id, які потрібно передавати в GET запити. Це дозволяє краще розділяти відповідальність за зміни/отримання даних, дозволяючи простіший контроль над тим, яка саме інформація повертається та полегшуючи тестування.
2. Деякі запити приймають тіла, в яких деякі поля ігноруються. Це пов'язано із тим, що вони використовують спільні dto із іншими endpoints. Створення додаткових dto потребуватиме синхронізації при змінах entity, натомість використання арі переважно відбувається іншими системами (фронт-ендом наприклад), де це не є суттєвою проблемою.
3. Endpoints можна в майбутньому обмежити роллю адміна, залишивши тільки пошук та бронювання доступними для інших.

Solution structure



1. Зв'язки між проектами

1. HallRentingService.Data – побудований на EF Core. Містить усі сутності та налаштування для бази даних. Використовується MS SQL база даних, яка запускається як окремий контейнер.
2. HallRentingService.ServiceDefaults – містить сервіси для dashboard від Aspire.
3. HallRentingService.WebAPI – сервер ASP.NET Core, де знаходиться уся логіка рішення. Pipeline обробки запитів має CQRS архітектуру.
4. HallRentingService.AppHost – запускає все рішення, зокрема контейнер із базою даних.
5. HallRentingService.UnitTests – проект із unit тестами. Переважно містить тести для FluentValidation.
6. HallRentingService.IntegrationTests – проект із інтеграційними тестами. Містить тести для handlers, із якими відбувається взаємодія через HttpClient. Для підготовки та перевірки роботи handlers, використовується DbContext, щоб взаємодіяти із базою даних.

Типи даних розбиті по-файлам та згруповані по features із дотриманням V-Slice архітектури.

Database structure, business task, and technical solutions



2. Діаграма бази даних

1. Керування конференц-залами – створено сутності HallEntity, HallServiceEntity, Hall_HallService_JoinEntity (M ⇔ N зв'язок) та CRUD для HallEntity та HallServiceEntity. В даній реалізації кожен конференц-зал може мати деякі послуги із загального списку послуг, але їхня ціна буде для кожного залу своя (тому Hall_HallService_JoinEntity має властивість “Price”).
2. Пошук та бронювання конференц-залів – створено сутність BookingEntity, яка має зв'язок із HallEntity (1 ⇔ N зв'язок), тобто HallEntity має список усіх бронювань. Пошук та бронювання залів можливий, якщо часовий проміжок бронювання не перетинається із іншими бронюваннями.
3. Вираховування ціни бронювання – кожен зал має ціну за годину і кожна послуга має власну ціну. До ціни залу, перед послугами застосовується знижка/націнка залежно від періоду дня бронювання. Якщо тривалість бронювання перетинає кілька проміжків знижок/націнок – застосовуються обидва відповідно до годин. Фінальна ціна зберігається в BookingEntity, що дозволяє змінювати ціни залу та послуг так, щоб це не впливало на активні бронювання.